

API TEST LAB

Self-hosted API testing, debugging, and performance experimentation platform

Project identity	A developer tool for running reproducible experiments against APIs
Primary goal	Build something personally useful across Node.js, Spring Boot, Dockerized, local, staging, and deployed APIs
V1 approach	Node.js + React, packaged as a Docker deployment
Long-term direction	Replace or augment the execution engine with C++ after benchmarking the Node implementation

Core principle

Optimize for usefulness + technical depth + coherent product design. A smaller product with excellent execution is better than a large feature checklist.

1. Why this project is worth building

The project is valuable because it is not intended to be another isolated portfolio application. It is a piece of personal developer infrastructure that can be used to test the other systems you build.

- Rate limiter: generate controlled concurrency and traffic to validate correctness, limits, latency, and behavior under pressure.
- Personal cloud server: exercise authentication, file uploads, large payloads, range requests, failures, and API workflows.
- Future full-stack / backend projects: save reusable collections and regression tests rather than manually testing endpoints every time.
- Portfolio value: demonstrate API design, concurrency, test execution, Docker, security isolation, data modeling, and performance measurement.

The important distinction

The project should not compete with mature API clients on request editing alone. Its identity should come from turning requests and workflows into reproducible experiments and then explaining what happened.

Working product definition: API Test Lab is a self-hosted developer platform for running, automating, and analyzing reproducible experiments against APIs.

2. Product model

Layer	Purpose	Examples
API interaction	Create and inspect requests	HTTP methods, headers, auth, bodies, files, response viewer
API experimentation	Change how requests are executed	Functional tests, workflows, load, payload, fuzz, resilience
Results	Explain and preserve outcomes	Assertions, latency, status distribution, history, comparison

This model is important because it keeps the product coherent. Load testing, fuzzing, and payload testing are not unrelated utilities; they are execution strategies applied to the same request/test/workflow model.

3. Recommended V1 scope

V1 should prove the core system without becoming a clone of every Postman feature.

API request client

GET/POST/PUT/PATCH/DELETE/HEAD/OPTIONS; query parameters; headers; cookies; authentication; JSON/raw; form-data; files; history; saved requests.

Environments & variables

Development/staging/production environments, reusable variables, secret values, and request interpolation such as `{{BASE_URL}}`.

Assertions & test runner

Status, response body, headers, response time, and JSON/schema checks. Produce explicit pass/fail results.

Workflows

Sequential request chains with response extraction and variables passed between steps.

Load testing

Request count, concurrency, requests/sec, duration, ramp-up/down, burst, and endurance modes; collect throughput, failures, latency, P50/P95/P99, timeouts, and status distribution.

Payload testing

Configurable JSON/text/file payload sizes and multipart uploads; test progressively larger payloads.

Test history & analytics

Persist runs, configurations, results, and basic comparisons so experiments are reproducible.

Docker deployment

Ship the whole application as a self-hosted deployment that developers open from their browser.

4. Features to consider later, not V1

Feature	Decision	Reason
Webhook Inspector	Later / likely worth it	Strong debugging utility; temporary endpoints, inspect, replay, modify, resend.
Fuzz / boundary testing	Later	Useful API-input experimentation, but keep scope smaller than a security fuzzing
Resilience / failure testing	Later	Useful for timeouts, malformed requests, retries, and unexpected responses once
CLI + CI/CD	Later / high priority	Turns saved collections into repeatable developer and CI workflows.
Mock API	Postpone	Useful but pulls product toward a different category; weaker fit than testing real A
Monitoring	Postpone	Can become a separate uptime-monitoring product; only add if clearly tied to test
Team/RBAC/cloud SaaS	Postpone	High complexity and does not improve the core self-hosted developer workflow.
Distributed cloud load testing	Postpone	Expensive and operationally complex; local/self-hosted execution is the better first

5. Deployment and architecture direction

The strongest early architecture is self-hosted and Docker-first. The web interface is the control plane; the test runner is the execution plane.

Conceptual V1

React Web UI -> Node.js / Express API -> Test Runner -> Target API

The user runs API Test Lab locally (for example through Docker) and opens the interface in a browser. The same runner can test local APIs, Dockerized services, staging APIs, and public online APIs.

Why self-hosted matters

- The user pays for and controls their own execution resources rather than making every load test an AWS cost for the product owner.
- Local API testing is naturally supported because the execution environment is close to the developer environment.
- Docker provides a consistent packaging model and a clear boundary for the runner and its dependencies.
- A future hosted version can still exist, but cloud execution should not be required for the core product.

Do not overengineer V1

Do not introduce Redis, RabbitMQ, Kubernetes, microservices, or multiple worker services simply to make the architecture look sophisticated. Add them only when a concrete scaling or reliability requirement appears.

6. Security is a first-class architecture problem

Because the system can send arbitrary HTTP requests and execute workloads selected by a user, the test runner is effectively a controlled network client with significant privileges.

Threat	What must be considered
SSRF	Prevent or control access to localhost, private network ranges, loopback, link-local addresses, cloud m
Resource exhaustion	Bound concurrency, duration, payload size, memory usage, CPU usage, open connections, and queued
File access	Do not allow arbitrary path traversal or unintended host-file access from file-upload features.
Secrets	Protect API keys, cookies, environment secrets, and authentication data in storage, logs, exports, and
Isolation	Define what the Docker runner can reach and whether it can communicate with the host or other cont
Abuse	Make load testing an explicit, bounded operation rather than an unrestricted public service.
Webhooks	Treat incoming webhook content as untrusted and isolate it from privileged internal resources.

7. Node.js first, C++ later

Node.js should be the V1 implementation because it matches the current skillset and is fully capable of building the product and proving the model.

V1 execution path

React -> Node/Express -> Node test runner -> target API -> result aggregation -> PostgreSQL

The key design decision is to treat the runner as a separate subsystem conceptually, even if it initially lives in the same application or deployment.

V2 performance path

React -> Node/Express control plane -> C++ test engine -> target API -> aggregated metrics -> Node/PostgreSQL

Do not rewrite in C++ simply because C++ is faster. First benchmark the Node runner. If Node becomes the limiting factor in high-concurrency or high-throughput workloads, replace the execution-heavy component with C++ and compare both engines under controlled tests.

8. What makes the project technically strong

- Concurrency: controlled parallel request execution and accurate timing.
- Networking: HTTP behavior, connection handling, timeouts, retries, and failure modes.
- Distributed-systems concepts: backpressure, worker coordination, job execution, and result aggregation when the system grows.
- Security: SSRF prevention, isolation, secret handling, and abuse controls.
- Data modeling: requests, environments, collections, workflows, runs, metrics, and historical results.
- Developer tooling: Docker packaging, reusable collections, and eventually a CLI/CI interface.
- Performance engineering: P50/P95/P99 measurement, load generation, benchmarking, and a justified future native execution engine.

Portfolio positioning

The project becomes especially strong when it is demonstrably used as personal infrastructure rather than presented as a one-off demo. The story is: build a rate limiter, cloud server, and future APIs; then build a testing laboratory that exercises those systems under real functional, payload, concurrency, and failure scenarios.

That creates a connected portfolio rather than several unrelated applications.

9. Current decisions to carry forward

- Product identity: self-hosted API testing + experimentation platform.
- Primary differentiator: reproducible experiments and analysis, not basic HTTP request editing.
- Deployment: Docker-first; users run the tool themselves.
- Execution: local/self-hosted runner is the foundation; hosted execution is optional later.
- V1 technology: React + Node.js/Express + PostgreSQL, with a modular test-runner boundary.
- Infrastructure rule: no Redis/RabbitMQ/microservices unless a real requirement appears.
- Load testing: important product capability, but introduced after the core execution/test model works.

- Mock API: postpone because it weakens product focus relative to real API experimentation.
- C++: future execution-engine optimization justified by benchmarks, not assumptions.

Current scope boundary

The project is being discussed and designed before implementation. The objective is to refine the product and architecture first, then implement a focused V1.